



DOCUMENTO DE PLATAFORMA

PLATAFORMA DE ENTREGA Y OPERACIÓN

Cómo se construye, se verifica, se despliega y se opera el producto: ramas y promoción, integración continua, infraestructura como código, secretos, observabilidad, objetivos de servicio y recuperación ante desastres.

PRODUCTO	VERSIÓN	FECHA	AUTOR	ESTADO
ZonaRafaga	1.0	01/09/2026	Luis Alfonso Hernández Núñez	Vigente

■ **CONTENIDO**

1	Propósito y alcance
2	Estrategia de ramas y promoción
3	Integración continua y empaquetado
4	Infraestructura como código
5	Gestión de secretos
6	Observabilidad
7	Objetivos de nivel de servicio
8	Recuperación ante desastres
9	Riesgos operativos

ESTADO DE LO AQUÍ DESCRITO

Las secciones 2 y 3 describen lo que **ya opera**. Las secciones 4 a 8 describen el diseño de plataforma **declarado y validado, sin recursos provisionados**: las plantillas compilan y su plan de cambios se revisa en cada entrega, pero no hay ambientes vivos. Esa distinción se sostiene en todo el documento y se declara en cada sección donde aplica.

1 PROPÓSITO Y ALCANCE

Este documento cubre todo lo que rodea al código: cómo llega a producción, cómo se sabe que funciona y qué se hace cuando deja de funcionar.

PRINCIPIO RECTOR

La infraestructura se **declara en código y se valida en cada entrega; no se provisiona**. Un ambiente existe cuando su plantilla compila, su plan de cambios se puede revisar y el pipeline lo publicaría sin intervención manual. Provisionarlo es una decisión de negocio, no un requisito técnico.

La separación es deliberada: permite construir y auditar la cadena completa —construcción, calidad, empaquetado y despliegue— sin pagar cómputo ocioso mientras el producto no tiene clientes.

LO QUE ESTO NO ES

Un plan de cambios validado **no es un despliegue probado**. Fallos de permisos, cuotas y dependencias entre recursos solo aparecen al aplicar. Se declara como limitación conocida en lugar de presentar el plan como si fuera un despliegue exitoso.

SERVICIOS EN USO

SERVICIO	ESTADO	PARA QUÉ
Repositorio y Pull Requests	EN USO	Código y revisión, con GitFlow.
Tablero de trabajo	EN USO	Requisitos, estimación y horas reales.
Pipelines	EN USO	Integración continua y empaquetado, sobre agente propio.
Registro de contenedores	EN USO	Imágenes por versión, identificadas por digest.
Infraestructura como código	DECLARADA	Terraform: validada y planificada, no aplicada.
Ambientes provisionados	NO EXISTEN	—

2 ESTRATEGIA DE RAMAS Y PROMOCIÓN

Tres ramas permanentes, una por ambiente. La rama es el mecanismo de promoción.

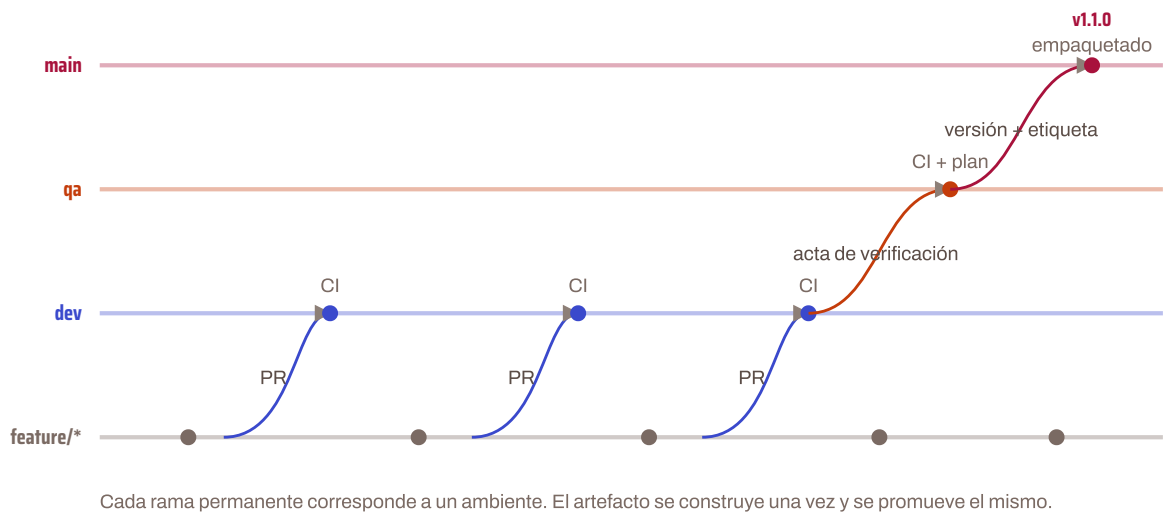


FIGURA 1 Camino de un cambio: nace en una rama de trabajo, entra a integración por Pull Request, se promueve a verificación con su acta y llega a producción con su etiqueta de versión.

POR QUÉ GITFLOW CON UN SOLO DESARROLLADOR

Por tamaño de equipo bastaría una sola rama, y el principio rector del proyecto es que la solución más simple gana. Se usan tres porque **sin ellas no hay dónde verificar una entrega candidata antes de que llegue a producción**, ni evidencia de promoción entre ambientes. Es una excepción consciente, y se sostiene solo mientras existan los tres ambientes.

RAMA	CONTIENE	QUIÉN LA ESCRIBE
main	Lo que está en producción. Cada punto es una versión etiquetada.	Solo Pull Request desde qa o desde una corrección urgente.
qa	La entrega candidata, en verificación.	Solo Pull Request desde dev .
dev	Trabajo terminado y validado, aún sin entregar.	Solo Pull Request desde ramas de trabajo.
feature/*	Una unidad de trabajo.	Nace de dev y muere ahí.
hotfix/*	Corrección urgente de producción.	Nace de main , vuelve a main y se reintegra a dev en el mismo turno.

REGLAS QUE SOSTIENEN LA TRAZABILIDAD

- **Nadie escribe directo en una rama permanente**, ni trabajando solo. El Pull Request es donde viven la revisión, los gates y el enlace al requisito.
- **El artefacto se construye una vez** y se promueve el mismo, identificado por digest. Reconstruir por ambiente es desplegar algo distinto de lo que se verificó.
- **Se mezcla con merge commit, nunca aplastado** entre ramas permanentes: aplastar destruye la correspondencia entre lo verificado y lo entregado.

- Un hotfix que no vuelve a **dev** reaparece en la siguiente entrega. Es la falla clásica de este modelo y por eso la reintegración es parte del mismo turno.

3 INTEGRACIÓN CONTINUA Y EMPAQUETADO

Qué corre en cada Pull Request y qué produce.

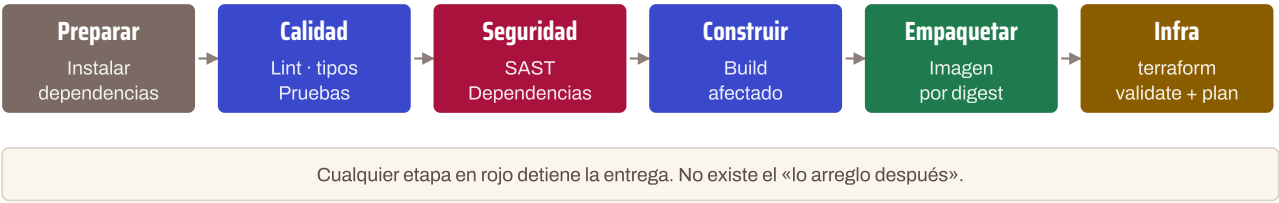


FIGURA 2 Etapas de la entrega. Las primeras cuatro corren en cada Pull Request; empaquetado e infraestructura solo al promover.

LOS GATES

Se ejecutan **solo sobre lo afectado** por el cambio, no sobre todo el repositorio. Un monorepo que reconstruye entero en cada cambio deja de ser usable en cuanto crece.

GATE	UMBRAL	QUÉ ATRAPA
Análisis estático y formato	0 advertencias	Deriva de estilo, código muerto, restos de depuración.
Tipos	0 errores	Contratos rotos entre capas y entre paquetes.
Pruebas unitarias y de integración	Umbral de cobertura	Regresiones de comportamiento.
Aislamiento entre clientes	0 fugas	Tablas sin política, contexto mal fijado, escrituras cruzadas.
Análisis de seguridad del código	0 hallazgos	Inyección, secretos embebidos, criptografía mal usada.
Auditoría de dependencias	0 severidad alta	Vulnerabilidades conocidas de la cadena de suministro.
Extremo a extremo	Todos pasan	Que los recorridos críticos sigan funcionando de verdad.
Validación de infraestructura	Plan sin errores	Plantillas inválidas y cambios no intencionados.
Seguridad de la infraestructura	0 hallazgos	Puertos abiertos, cifrado ausente, registro apagado, secretos sin caducidad. Toda excepción lleva su motivo escrito junto al recurso.
Vulnerabilidades de la imagen	0 altas ni críticas	Paquetes del sistema y del intérprete con fallo conocido y corrección disponible. Se escanea antes de publicar: una imagen vulnerable ya en el registro es una imagen que alguien puede desplegar.
Inventario de componentes	Se emite siempre	Qué hay exactamente dentro de la imagen. Es lo que permite responder «¿nos afecta?» cuando salga la siguiente vulnerabilidad de una librería, sin adivinar.

DÓNDE CORRE

En **agentes efímeros**: cada etapa arranca en una máquina limpia y se destruye al terminar. Ninguna computadora del equipo es requisito para entregar, y una dependencia que alguien instaló a mano y nunca declaró falla en el pipeline en lugar de esconderse.

El precio es que el toolchain se declara aquí. La imagen no trae Terraform, Flutter, pnpm ni semgrep, así que el pipeline los instala en la **versión exacta** que fija `.toolchain` —el mismo archivo que lee el contenedor de desarrollo— y los restaura de caché: almacén de paquetes, caché de compilación, SDK y binarios.

OPCIÓN	USD/MES	QUÉ CUESTA DE VERDAD
Agentes efímeros	0.00	Un trabajo en paralelo con 1 800 minutos al mes y tope de 60 por trabajo.
Los mismos, de pago	40.00	Quita el tope mensual. Se compra cuando la cuota estorbe, sin rediseñar nada.
Agente propio en la nube	38.71	Cuesta casi lo mismo que el de pago y además hay que parchar el servidor y actualizar el agente. No hay escenario en que gane.

LA DEFINICIÓN DEL AGENTE PROPIO SE CONSERVA

No como el camino normal, sino como salida de emergencia para dos casos: que la cuota mensual se agote a media entrega, o que haga falta depurar una corrida con el espacio de trabajo intacto entre etapas —justo lo que un agente efímero no conserva—. Cambiar a él es modificar el pool en la definición, no reescribir el pipeline.

4 INFRAESTRUCTURA COMO CÓDIGO

Terraform con proveedor de Azure. Un módulo por recurso, un juego de variables por ambiente, y el mismo código para los tres.

POR QUÉ TERRAFORM Y NO LA HERRAMIENTA NATIVA

Para desplegar solo en Azure, la herramienta nativa es igual de buena y más simple: no administra estado. El desempate no es técnico sino de capacidad transferible — Terraform obliga a entender **estado, dependencias explícitas, plan contra aplicación y módulos reutilizables**, y eso sirve con cualquier proveedor. La decisión y su costo están registrados por separado.

ESTRUCTURA

ELEMENTO	CONTENIDO
<code>infra/modulos/</code>	Un módulo por recurso: red, base de datos, caché, cómputo, almacén de objetos, bóveda de secretos, registro, observabilidad. Cada uno con sus entradas, salidas y valores por omisión seguros —y el valor seguro es el cerrado: la red niega, la bóveda niega, el almacén niega, y se abre por subred.
<code>infra/ambientes/{dev,qa,prod}/</code>	La composición de módulos y las variables de ese ambiente. Lo único que cambia entre ambientes es el dimensionamiento y la redundancia, nunca la forma.
<code>infra/agent/</code>	La imagen del agente de emergencia, versionada con el resto. Lee las mismas versiones del toolchain que el pipeline, así que no puede quedar desalineada.

EL ESTADO: LA PRIMERA DECISIÓN, NO LA ÚLTIMA

Sin estado remoto el proyecto no es reproducible: el estado vivirá en la máquina de quien aplicó el último cambio. Se resuelve con almacenamiento remoto por ambiente, con **bloqueo** para impedir dos aplicaciones simultáneas y **versionado** para poder volver atrás.

EL ESTADO GUARDA SECRETOS EN CLARO

Cadenas de conexión, llaves y contraseñas generadas quedan legibles dentro del archivo de estado. No es un detalle de operación: es parte del diseño. Exige cifrado en reposo, acceso restringido a quien aplica, y **nunca** subirlo al repositorio. Es la diferencia más importante frente a una herramienta sin estado.

QUÉ CORRE EN CADA ENTREGA

- 1. **Formato y validación** de las plantillas: atrapa errores de sintaxis y de tipos.
- 2. **Análisis estático de seguridad** sobre la infraestructura: puertos abiertos, almacenamiento público, cifrado ausente.
- 3. **Plan de cambios** contra el ambiente destino, publicado como artefacto de la ejecución para poder revisarlo.

Ninguna entrega aplica cambios. Aplicar es una acción deliberada, fuera del pipeline, el día que exista un ambiente.

5 GESTIÓN DE SECRETOS

Ningún secreto vive en el repositorio, en el pipeline ni en una imagen.

DÓNDE	CÓMO SE GUARDA	CÓMO LLEGA A QUIEN LO USA
Local	Archivo de entorno, fuera del control de versiones.	Lo lee el proceso al arrancar. Existe un ejemplo versionado, sin valores.
Pipeline	Variables protegidas del propio servicio, o referencia a la bóveda.	Se inyectan solo en la etapa que las necesita, nunca en el registro de ejecución.
Agente	Secreto de contenedor montado desde archivo.	Solo lectura, dentro del contenedor. No aparece al inspeccionarlo.
Ambiente	Bóveda de secretos; la infraestructura declara la <i>referencia</i> , no el valor.	La aplicación los resuelve con su identidad administrada, sin credencial propia.

REGLAS

- **Las plantillas declaran referencias, no valores.** Un secreto escrito en una plantilla queda en el historial para siempre.
- **Rotación sin redespiegue.** Cambiar un secreto en la bóveda no debe exigir reconstruir la imagen; por eso se resuelve en tiempo de ejecución y no en tiempo de construcción.
- **Detección en el pipeline.** El análisis de seguridad incluye reglas de secretos; un secreto embebido rompe la entrega.
- **Los secretos no se registran.** Contraseñas y tokens se excluyen de los registros y de la auditoría; lo que se audita es el acceso, no el valor.

6 OBSERVABILIDAD

Tres señales, un objetivo: poder responder «¿está bien el sistema?» sin abrir el código.

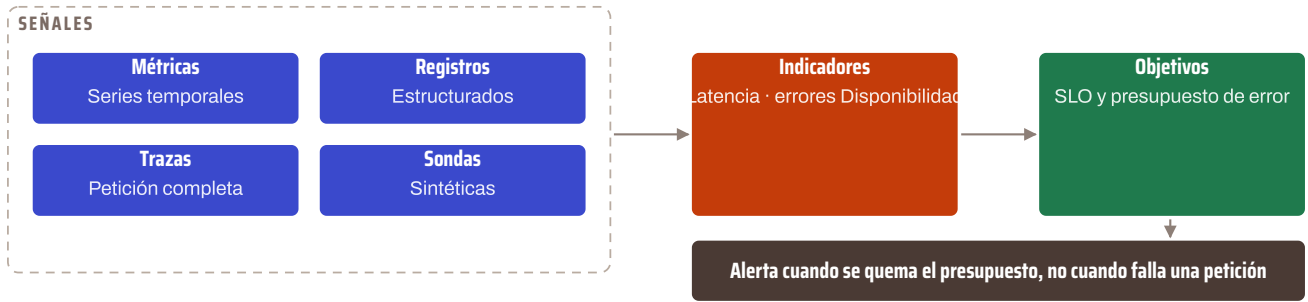


FIGURA 3 De señales a objetivos. Las señales por sí solas no dicen nada: se agregan en indicadores, y los indicadores se comparan contra un objetivo.

QUÉ SE INSTRUMENTA

SEÑAL	QUÉ SE CAPTURA	PARA QUÉ SIRVE
Métricas	Duración de petición por ruta y código, profundidad de cola de trabajos, eventos de partido por minuto, conexiones al canal en vivo.	Responder si el sistema está sano <i>ahora</i> y alimentar los indicadores.
Registros	Estructurados, con identificador de correlación, de liga y de usuario. Sin datos personales ni secretos.	Entender <i>qué pasó</i> en un caso concreto.
Trazas	La petición completa, de la entrada al almacén, incluyendo el paso por el canal en vivo y por los trabajos.	Encontrar <i>dónde</i> se fue el tiempo. Es lo único que sirve cuando algo está lento pero nada falla.
Sondas	Recorridos críticos ejecutados desde fuera cada pocos minutos.	Enterarse antes que el usuario, y medir disponibilidad de verdad.

LO QUE YA EXISTE Y LO QUE FALTA

La aplicación **ya expone métricas** en formato estándar y verificaciones de vida y disponibilidad; los registros ya son estructurados. **Falta** quien las consuma —no hay recolector ni tablero— y falta la instrumentación de trazas. Sin ambiente vivo, no hay nada que observar todavía: la decisión es dejar la instrumentación lista y encender la recolección junto con el primer ambiente.

CORRELACIÓN, NO COLECCIÓN

Tres señales sin identificador común son tres silos. El identificador de correlación viaja desde el cliente en cada petición y aparece en la métrica, en el registro y en la traza. Es lo que convierte tres herramientas en una investigación.

7 OBJETIVOS DE NIVEL DE SERVICIO

Un objetivo de servicio no es una promesa de perfección: es un acuerdo sobre cuánta falla es tolerable, y qué se hace al agotarla.

INDICADORES Y OBJETIVOS

INDICADOR	OBJETIVO	PRESUPUESTO	VENTANA Y JUSTIFICACIÓN
Disponibilidad en partido Peticiónes correctas / totales	99.5 %	≈ 2 h/mes	Sábado y domingo 09:00–22:00. Es cuando se juega; fuera de esa ventana el objetivo baja a 99 % y abarata la operación.
Latencia del marcador Captura a pantalla	p95 < 2 s	5 % de eventos	El video de la liga trae 10–30 s de retraso; por debajo de 2 s el marcador va por delante.
Latencia de consulta pública	p95 < 800 ms	5 % de peticiones	Tabla de posiciones y ficha de jugador: por encima de un segundo se abandona.
Éxito de sincronización Eventos aceptados / enviados	99.9 %	1 de cada 1 000	Es el momento de la verdad del producto. Un evento perdido obliga a recapturar desde el acta en papel.

CÓMO SE USA EL PRESUPUESTO DE ERROR

- **Con presupuesto disponible**, se entrega. El presupuesto existe para gastarse en cambios, no para lucirlo intacto.
- **Con el presupuesto agotado**, se congelan las entregas de funcionalidad y el trabajo pasa a confiabilidad hasta recuperarlo. Es la regla que hace que el objetivo signifique algo.
- **Se alerta por consumo del presupuesto**, no por petición fallida. Alertar por cada error entrena a ignorar las alertas, que es peor que no tenerlas.

ESTOS OBJETIVOS AÚN NO SE MIDEN

Están derivados del dominio, no de tráfico observado: no hay ambiente vivo. Se convierten en umbrales reales cuando exista uno, y se corrigen contra el comportamiento medido. Presentarlos como cumplidos sería falso.

8 RECUPERACIÓN ANTE DESASTRES

Qué se pierde, cuánto se tarda en volver, y cómo se sabe que el respaldo sirve.

OBJETIVOS

OBJETIVO	META	JUSTIFICACIÓN
Pérdida máxima de datos	≤ 15 min	Perder más de un cuarto de partido obliga a recapturar desde el acta en papel, y el acta puede no existir.
Tiempo de restauración	≤ 4 h	Permite recuperar dentro de la misma jornada de partidos de fin de semana.

OBJETIVO	META	JUSTIFICACIÓN
Retención de respaldos	35 días	Cubre una temporada corta completa y el tiempo típico en que se detecta una corrupción de datos.

QUÉ SE RESPALDA Y CÓMO

ACTIVO	MECANISMO	NOTA
Base de datos	Respaldo automático con restauración a un punto en el tiempo	Es lo único irremplazable. Todo lo demás se puede reconstruir.
Almacén de objetos	Versionado y borrado suave	Protege contra borrado accidental, no solo contra falla de disco.
Estado de infraestructura	Versionado del almacenamiento remoto	Un estado corrupto deja la infraestructura inmanejable aunque los recursos sigan vivos.
Imágenes y artefactos	Retención por versión en el registro	Permite volver a una versión anterior sin reconstruir.
Código y configuración	El propio repositorio	Con historial completo y ramas permanentes.

LA REGLA QUE HACE REAL TODO LO ANTERIOR

Un respaldo que no se ha restaurado no es un respaldo: es una suposición. El procedimiento de restauración se ejecuta y se cronometra al menos una vez por temporada, sobre un ambiente desechable, y su resultado se anota junto al objetivo de tiempo. Si tarda más de lo prometido, se corrige el objetivo o se corrige el procedimiento — pero no se deja el número bonito.

PENDIENTE HASTA QUE EXISTA UN AMBIENTE

Los mecanismos están declarados en las plantillas de infraestructura; ninguno se ha ejercitado, porque no hay nada que respaldar. La primera prueba de restauración es parte del trabajo de encender el primer ambiente, no algo posterior.

9 RIESGOS OPERATIVOS

RIESGO	IMPACTO	MITIGACIÓN
La cuota mensual de minutos se agota	BAJO	Con el consumo estimado entran del orden de veinticuatro unidades de trabajo al mes. Si estorba, se levanta el tope por USD 40 al mes; entre tanto queda la salida del agente propio, que no tiene límite de minutos.
El estado de Terraform se corrompe o se pierde	ALTO	Deja la infraestructura inmanejable aunque siga corriendo. Almacenamiento remoto con versionado y bloqueo; recuperación probada junto con la restauración de la base de datos.
Un secreto entra al repositorio	ALTO	Queda en el historial para siempre. Reglas de detección de secretos en el análisis estático, que rompen la entrega; archivos de secretos fuera del control de versiones desde el inicio.
Los objetivos de servicio son inventados	MEDIO	Sin tráfico real pueden estar mal calibrados. Se declaran como propuestos y se corrigen con la primera medición, en lugar de defenderlos.

RIESGO	IMPACTO	MITIGACIÓN
Un solo desarrollador aprueba sus propios cambios	MEDIO	La revisión por pares no existe. El valor lo aportan los gates automáticos y la trazabilidad al requisito, no la aprobación. Se declara como limitación, no se disfraza.

QUÉ HARÍA FALTA PARA OPERAR DE VERDAD

Encender un ambiente exige, en este orden: estado remoto de infraestructura, bóveda de secretos con identidad administrada, recolección de observabilidad, y la primera prueba de restauración. Hasta que esos cuatro existan, lo que hay es una plataforma diseñada y validada, no operada.